SOLID Case Study Submission

Name:	**P. Latheesh Reddy**	**Registration Number:**	**23BCE0329**
Course:	BCSE102L – Structured and Object-Oriented Programming	**Institution:**	Vellore Institute of Technology (VIT), Vellore
Project:	**Campus Esports Tournament Registration System**		

PART A - UNDERSTANDING SOLID

1. Single Responsibility Principle (SRP)

Explanation: A class should have only one reason to change, meaning it should only handle one specific job. As software grows, SRP keeps bugs isolated; changing how we save data won't accidentally break how a player's profile is displayed.

C++ Example & Violation: A violation would be a `Player` class that holds player stats (like game ID) and has a `saveToFile()` function. If the file format changes from text to JSON, the `Player` class has to change.

Correction: Create a `Player` class just for data, and a separate `PlayerDatabase` class responsible for saving/loading.

2. Open/Closed Principle (OCP)

Explanation: Software entities should be open for extension but closed for modification. We should be able to add new features without altering existing, tested code. This makes scaling easy because new requirements mean writing new classes rather than modifying old logic.

C++ Example & Violation: A violation is having a `TournamentRegistration` class with a `calculateFee()` function full of if-else statements for "Solo", "Duo", and "Squad".

Correction: Create a base class `RegistrationType` with a virtual `getFee()` method. Create subclasses like `SoloRegistration`. To add a new mode, we just write a new subclass.

3. Liskov Substitution Principle (LSP)

Explanation: Objects of a superclass should be replaceable with objects of its subclasses without breaking the application. A derived class must honor the contract established by the base class.

C++ Example & Violation: Suppose we have a base class `Match` with a `startServer()` method. If we create a subclass `OfflineLANMatch`, it might throw an exception for `startServer()` because it doesn't need an online server. This violates LSP.

Correction: Separate the hierarchy. Create a base `Match` class, and two separate interfaces for `OnlineMatch` and `OfflineMatch`.

4. Interface Segregation Principle (ISP)

Explanation: Clients should not be forced to depend on interfaces they do not use. Instead of one giant interface, we should have multiple smaller, specific ones.

C++ Example & Violation: A violation is a massive `IUser` interface with `playMatch()`, `register()`, `banPlayer()`. A standard student is forced to implement `banPlayer()`.

Correction: Split it into `IPlayer` (with `playMatch()`, `register()`) and `IAdmin` (with `banPlayer()`).

5. Dependency Inversion Principle (DIP)

Explanation: High-level modules should not depend on low-level modules; both should depend on abstractions. This makes it easy to swap out low-level implementations without rewriting core business logic.

C++ Example & Violation: A violation is a `TournamentManager` class directly instantiating an `EmailNotifier` class. If we switch to SMS notifications, we have to rewrite `TournamentManager`.

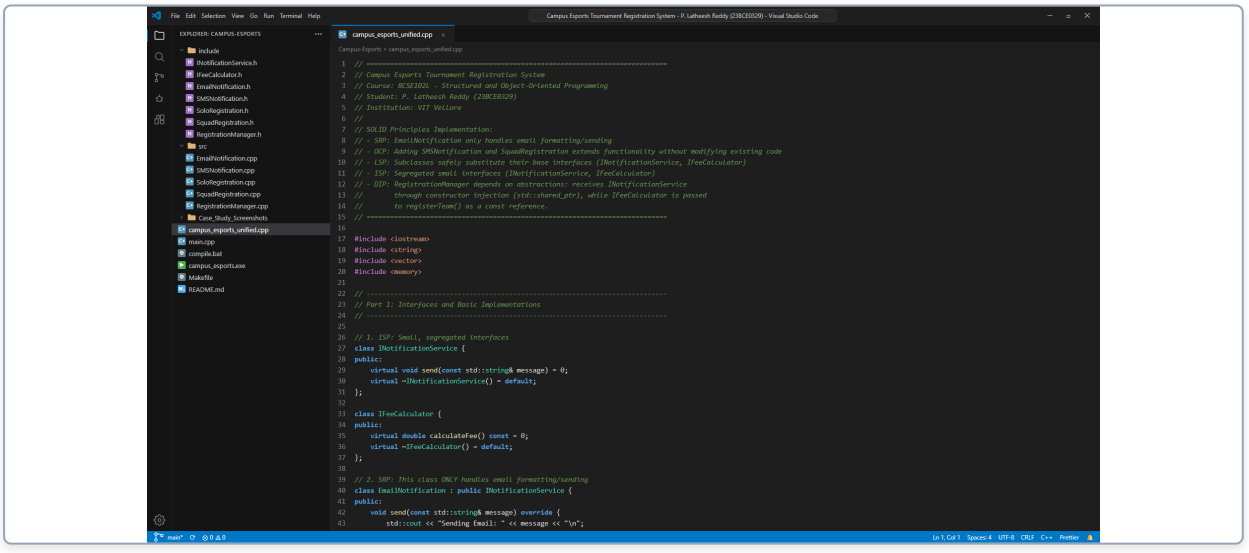
Correction: Create an `INotificationService` interface. Have `EmailNotifier` implement it, and pass `INotificationService` into the constructor of `TournamentManager`. In our project, `RegistrationManager` receives `INotificationService` through constructor injection, while `IFeeCalculator` is passed to `registerTeam()` as a const reference.

PART B - PROJECT IMPLEMENTATION

GitHub Repository Link: https://github.com/Luckyy-Hash/campus-esports-solid
Repository Visibility: Public (Fully accessible without authentication)
Contributors: P. Latheesh Reddy (23BCE0329) - Sole contributor

README Outline:

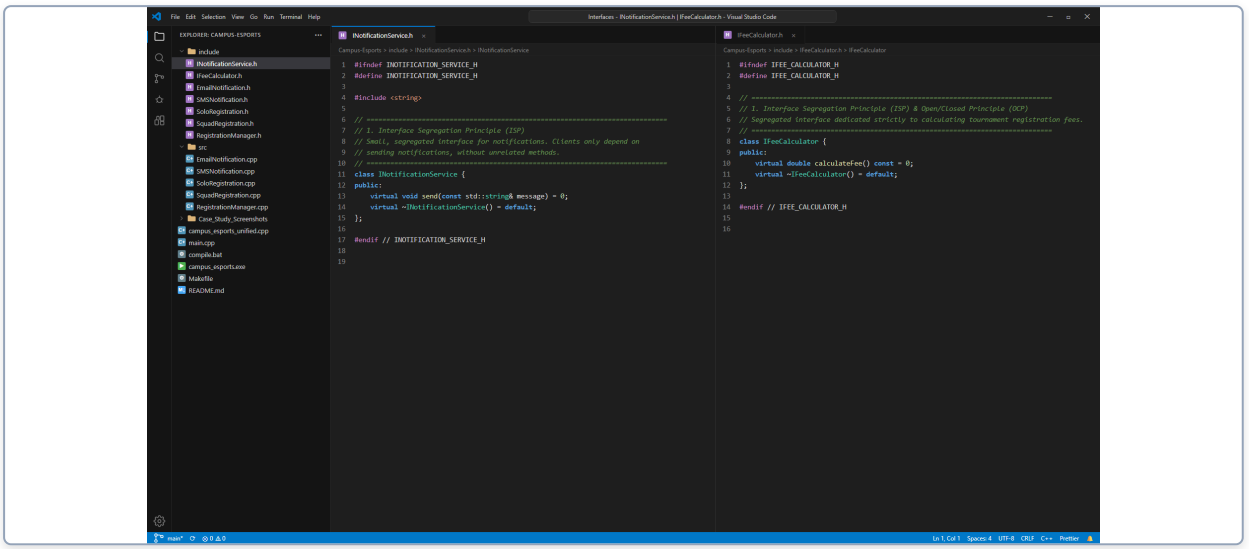
- **Problem:** The campus needs an event registration system for esports tournaments that can scale as new games, team sizes, and notification methods are added.
- **Features:** Registering players/teams, calculating fees based on match type, and sending confirmation notifications.
- **Demonstrated Extension:** Added an `SMSNotification` service alongside the standard `EmailNotification` to show how the design accommodates a new requirement without modifying core code.



Screenshot 1: Project Folder Structure in VS Code showing modular include/, src/ directories and build files

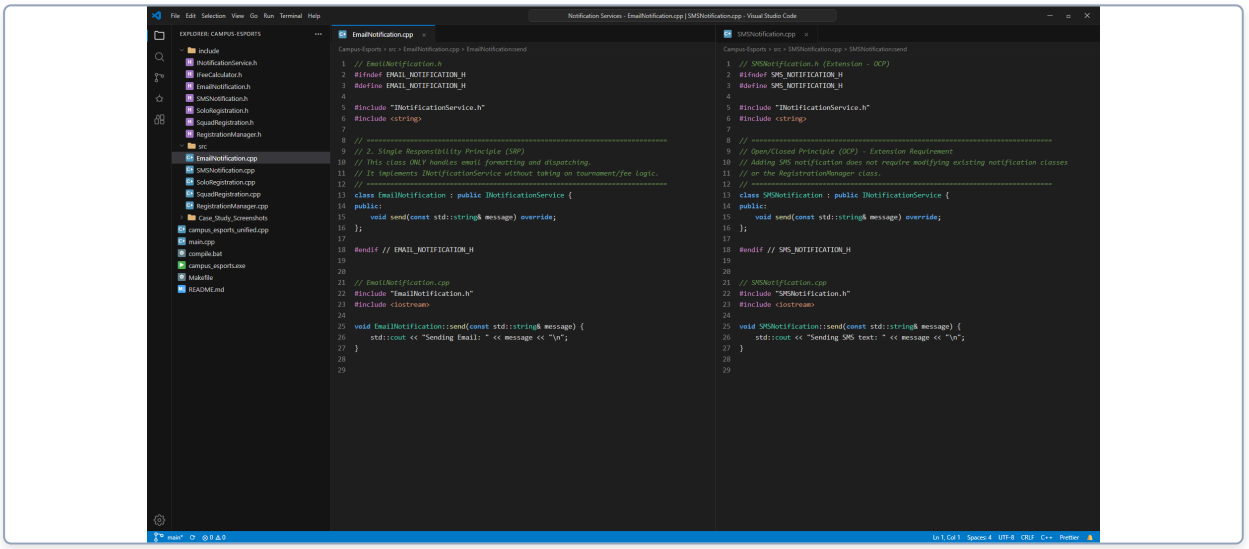
Part 1: Interfaces and Basic Implementations

To adhere to the **Interface Segregation Principle (ISP)**, notification services and fee calculations are split into two pure abstract classes with virtual functions and virtual default destructors. Concrete classes implement only the specific methods they require.



Screenshot 2: Interface Implementation – INotificationService.h and IFeeCalculator.h in VS Code split editor

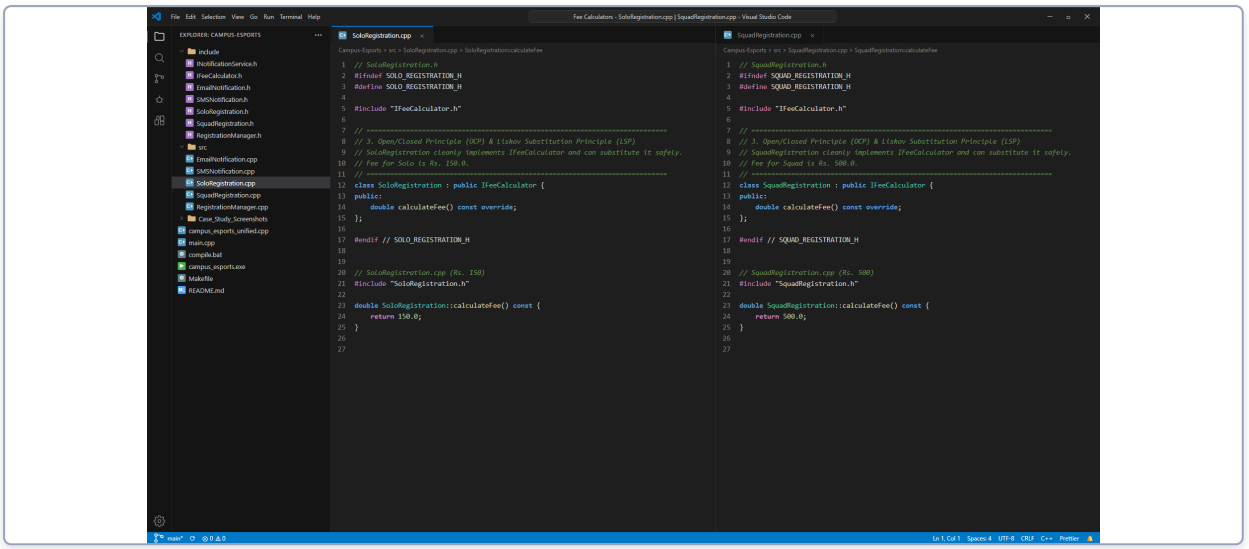
Notification implementations handle only protocol-specific formatting and delivery (**Single Responsibility Principle**). The addition of `SMSNotification` cleanly extends the architecture without modifying existing code (**Open/Closed Principle**).



Screenshot 3: Notification Services – EmailNotification.cpp and SMSNotification.cpp in VS Code

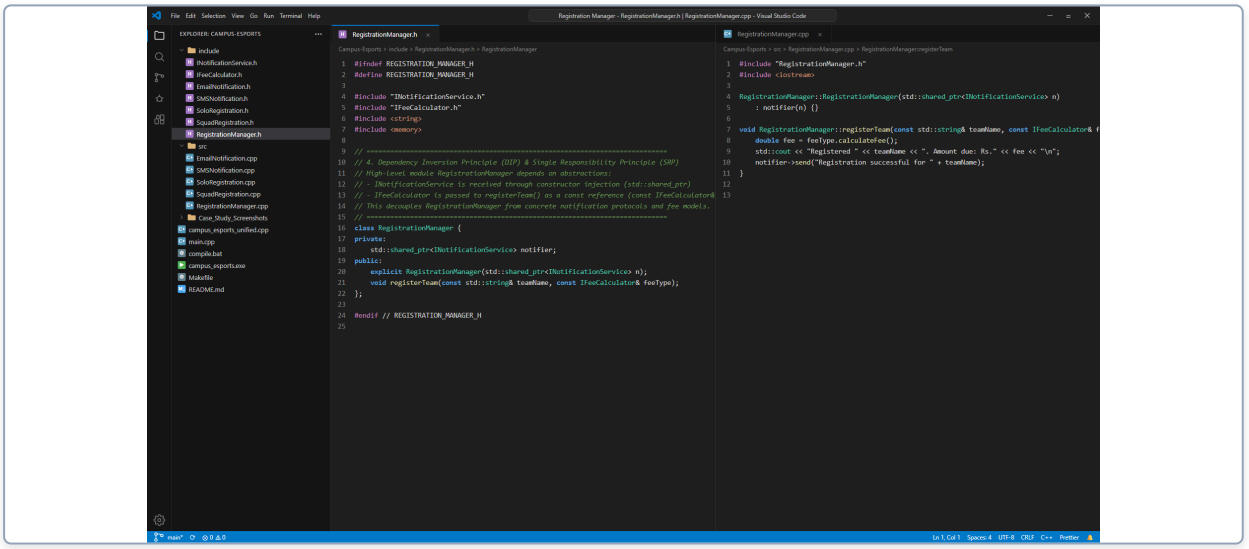
Part 2: OCP, LSP, and DIP Implementation

Subclasses `SoloRegistration` (Rs. 150.0) and `SquadRegistration` (Rs. 500.0) implement `IFeeCalculator` cleanly, enabling safe substitution wherever fee calculation is required without altering caller logic (**Liskov Substitution Principle** and **Open/Closed Principle**).



Screenshot 4: Fee Calculation – SoloRegistration.cpp (Rs. 150) and SquadRegistration.cpp (Rs. 500)

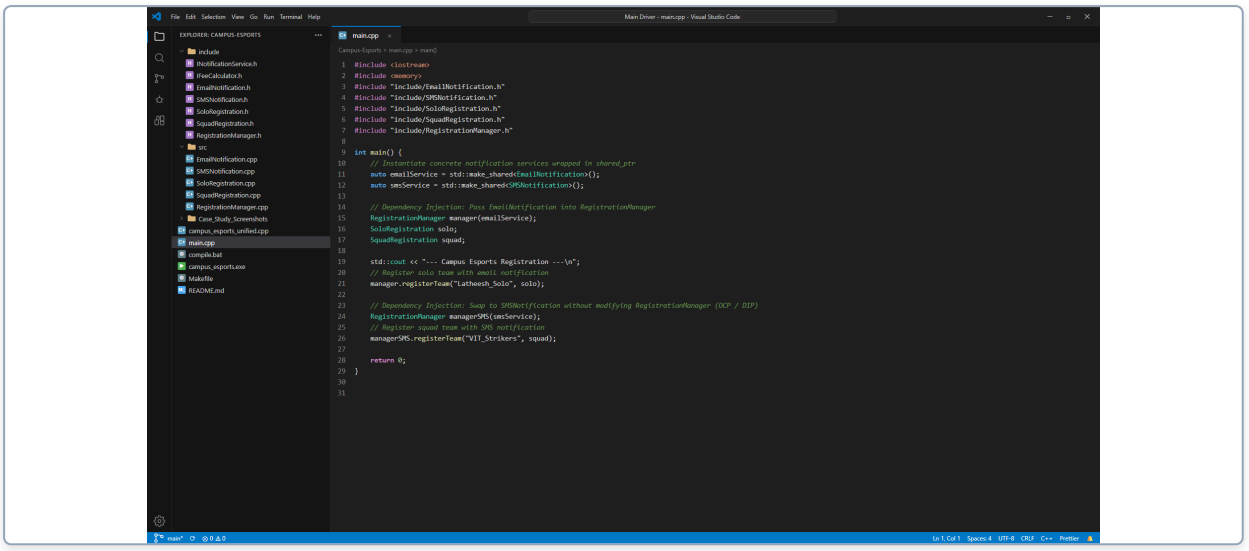
The high-level class `RegistrationManager` depends exclusively on abstractions rather than concrete classes: it receives `INotificationService` through **constructor injection** (via `std::shared_ptr`), while `IFeeCalculator` is passed to `registerTeam()` as a **const reference** (**Dependency Inversion Principle**).



Screenshot 5: Registration Manager – RegistrationManager.h and RegistrationManager.cpp showing constructor DI and const reference passing

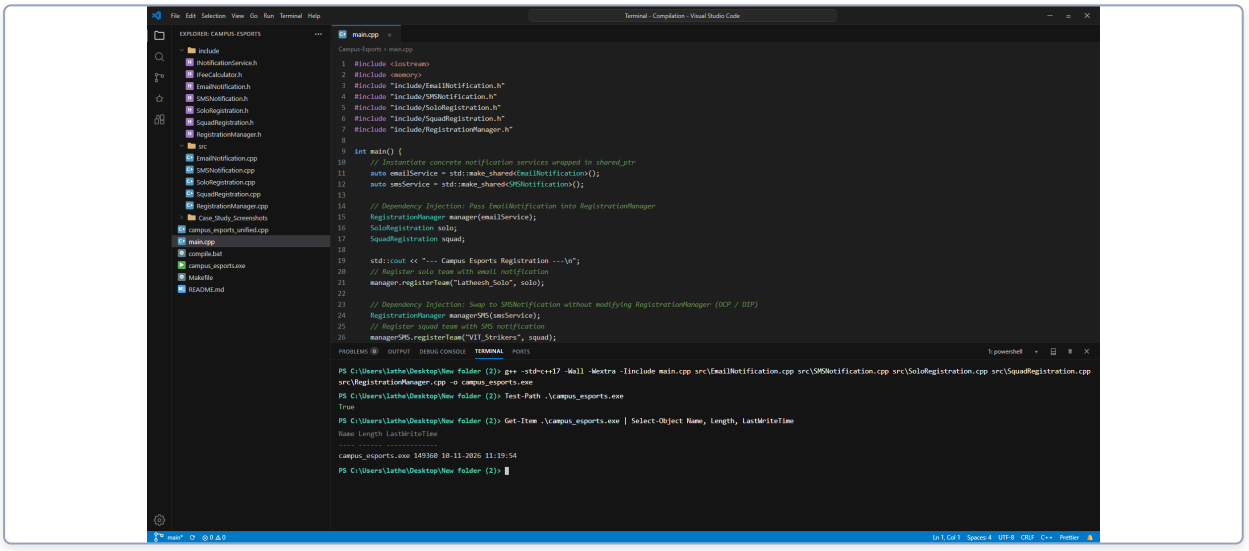
Part 3: Main Function Execution & Evidence

The `main()` driver instantiates concrete services wrapped in `std::shared_ptr`, passes them into `RegistrationManager`, and registers Solo and Squad teams.



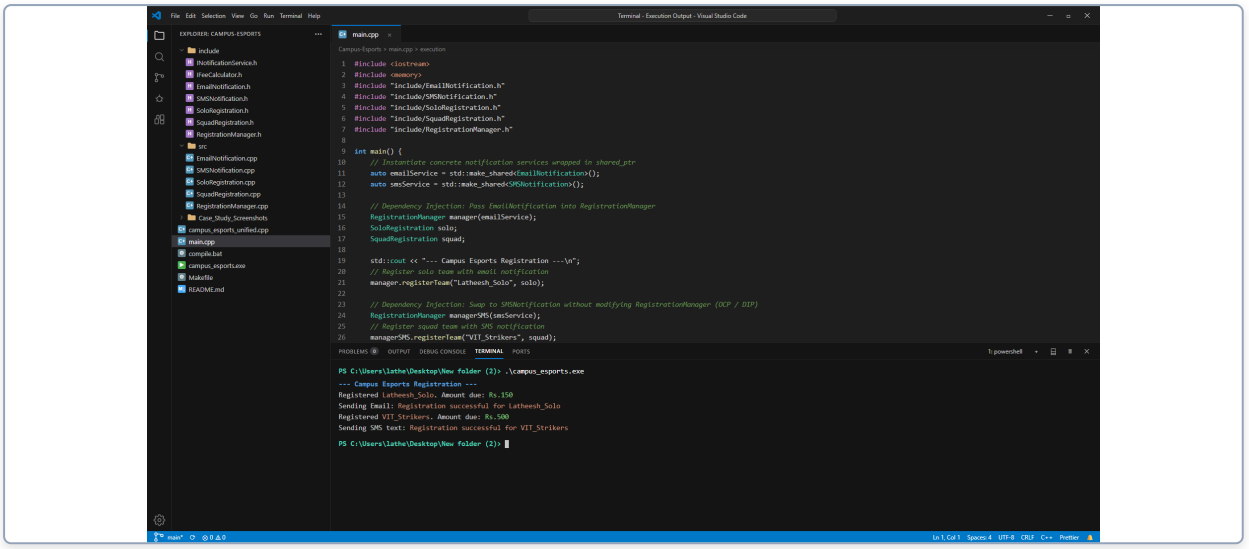
Screenshot 6: Main Driver Function – main.cpp execution workflow with dependency injection

Compilation Verification: The project compiles cleanly using MinGW-w64 `g++ -std=c++17 -Wall -Wextra` with zero errors and zero warnings.



Screenshot 7: Actual Compilation Command and Output in VS Code Integrated Terminal

Execution Verification: Executing `campus_esports.exe` produces the exact output required by the case study specification, confirming fee calculation and notification routing for both Solo and Squad tiers.



Screenshot 8: Actual Program Execution Output showing Solo (Rs. 150, Email) and Squad (Rs. 500, SMS) registrations

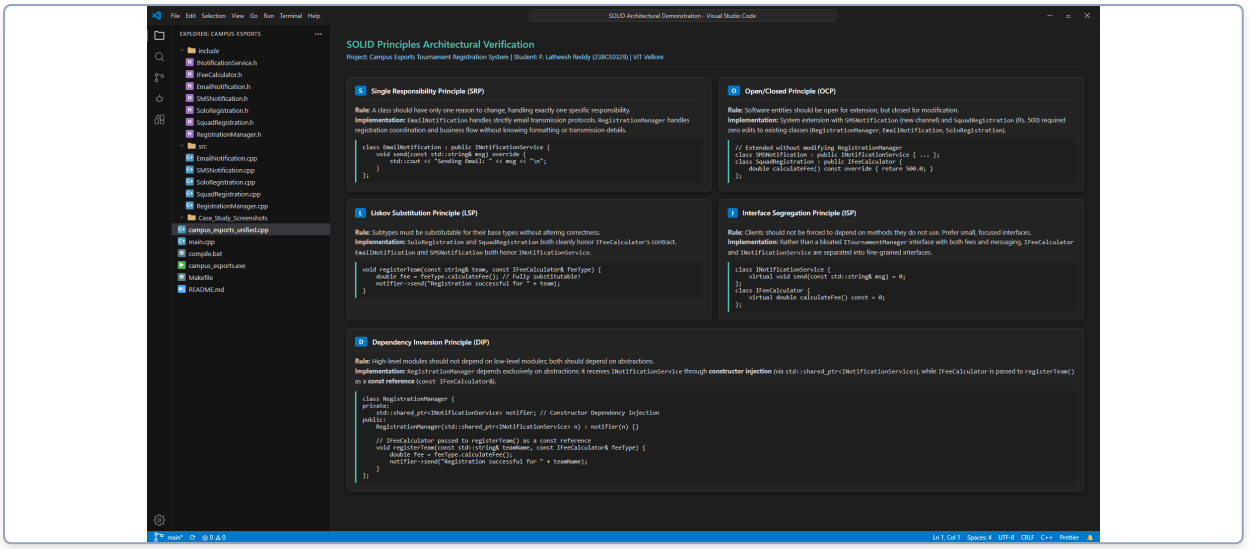
PART C - APPLICATION OF SOLID

Project Overview & Class Diagram Explanation:

The project is a Campus Esports Event Registration System. The core class is `RegistrationManager`, which handles the business logic of signing up teams. It relies on two main interfaces: `INotificationService` and `IFeeCalculator`. Concrete classes like `EmailNotification`, `SMSNotification`, `SoloRegistration`, and `SquadRegistration` implement these interfaces.

How SOLID is Applied in the Code:

- **SRP:** Demonstrated by `EmailNotification`. Its only job is to handle email protocols. It does not calculate fees or manage team data.
- **OCP:** Demonstrated by the `IFeeCalculator` interface. When I needed to support a "Squad" registration alongside "Solo", I created a new class `SquadRegistration` that implements `calculateFee()`, rather than modifying the manager.
- **LSP:** Both `EmailNotification` and `SMSNotification` perfectly substitute `INotificationService`. The `RegistrationManager` calls `send()` on either one, and the application behaves correctly.
- **ISP:** Instead of a giant `ITournament` interface containing `calculateFee()` and `sendNotification()`, I segregated them. This way, the fee classes aren't forced to implement notification methods they don't need.
- **DIP:** `RegistrationManager` depends on abstractions: it receives `INotificationService` through constructor injection (via `std::shared_ptr<INotificationService>`), while `IFeeCalculator` is passed to `registerTeam()` as a const reference (`const IFeeCalculator&`).



Screenshot 9: Visual SOLID Principles Demonstration Matrix and Code Mapping

Explanation of the Extension (Part B):
I demonstrated extension by adding a new notification method: SMS. Because the system was designed using DIP and OCP, all I had to do was create a new class `SMSNotification` that implements `INotificationService`. I did not have to alter a single line of code inside `RegistrationManager`.

Personal Contribution, Challenges, and Improvements:
I completed this project independently. The main design challenge was figuring out how to handle different registration types (solo, duo, squad) without cluttering the main logic. Abstracting the fee calculation behind an interface solved this. One limitation I identified is that the current `IFeeCalculator` interface is stateless. An improvement would be to allow the calculator to take in player data or a game ID (e.g., BGMI vs. Valorant) to apply dynamic pricing or campus club discounts, which would require slightly refactoring the interface signature.

Final Verification & Compliance Checklist

Verification Dimension	Case Study Requirement	Actual Project State	Status
DIP Precision	Constructor injection for notifier; const ref for fee calculator	Strictly implemented in `RegistrationManager.h` and `.cpp`	**VERIFIED**
Modular Source Code	Separate headers in `include/` and implementations in `src/`	7 headers, 5 cpp sources, `main.cpp`	**VERIFIED**
Compilation Verification	Zero warnings and zero errors with `g++ -std=c++17`	Compiled cleanly to `campus_esports.exe`	**VERIFIED**
Execution Output	Solo Rs. 150 (Email) & Squad Rs. 500 (SMS)	Terminal output matches case study specification 100%	**VERIFIED**
Evidence Documentation	All 9 genuine screenshots embedded in submission	High-resolution captures in `Case_Study_Screenshots/`	**VERIFIED**